САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа №4

Выполнил:

Максимова Ольга

Группа K3342

Проверил:
Добряков Д. И.

Санкт-Петербург

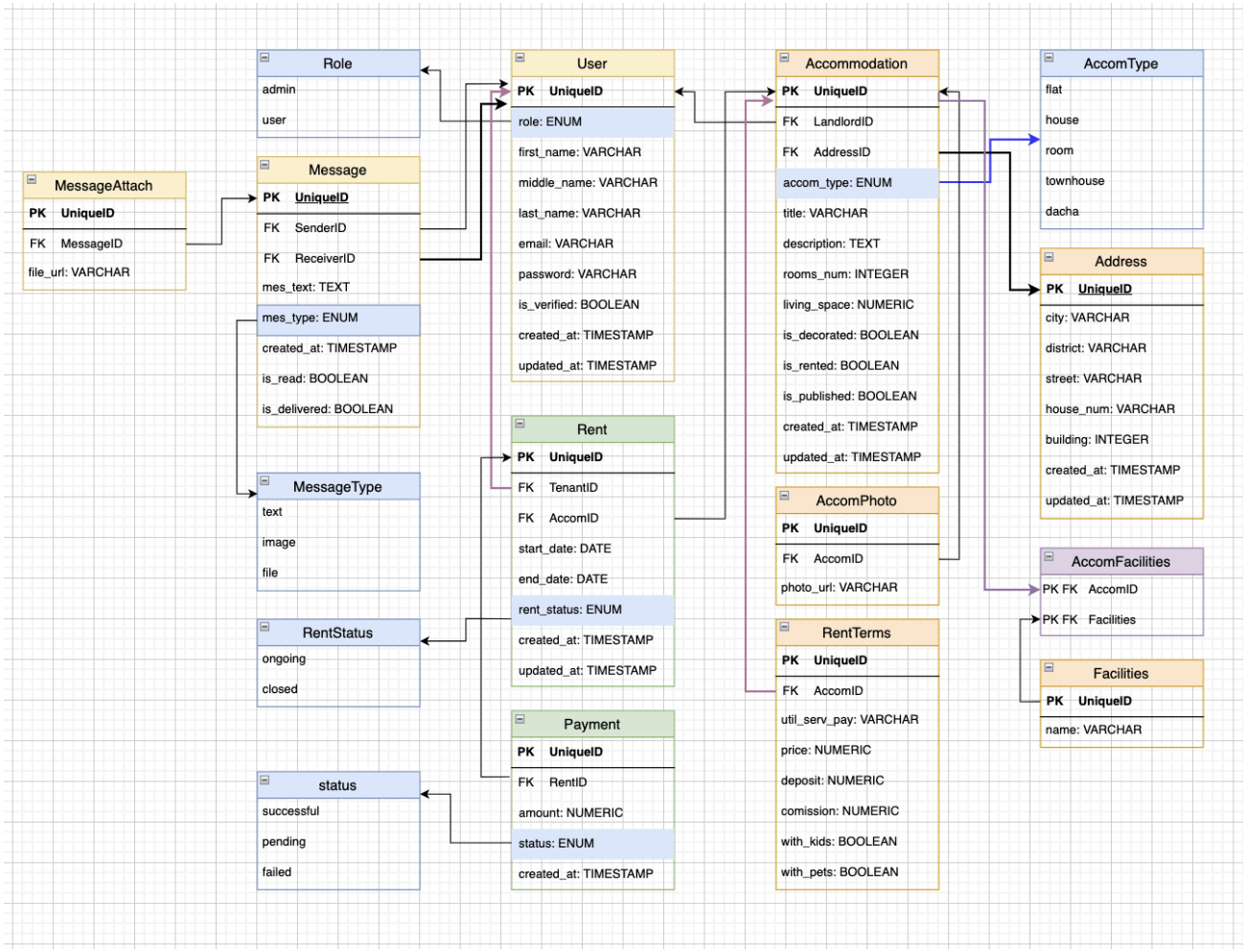
2026 г.

Задача

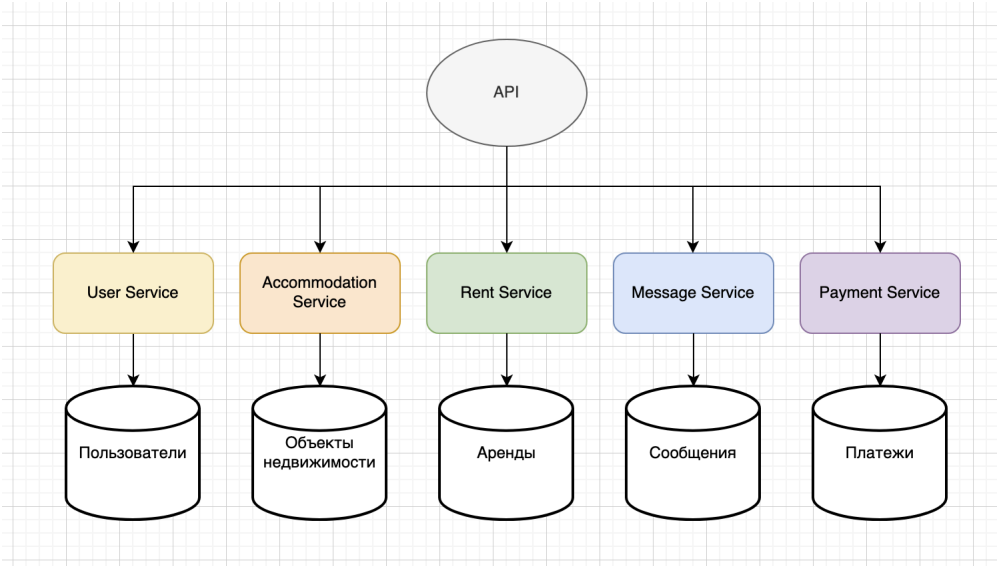
1. Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разеделения вашего монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

Текущая схема:

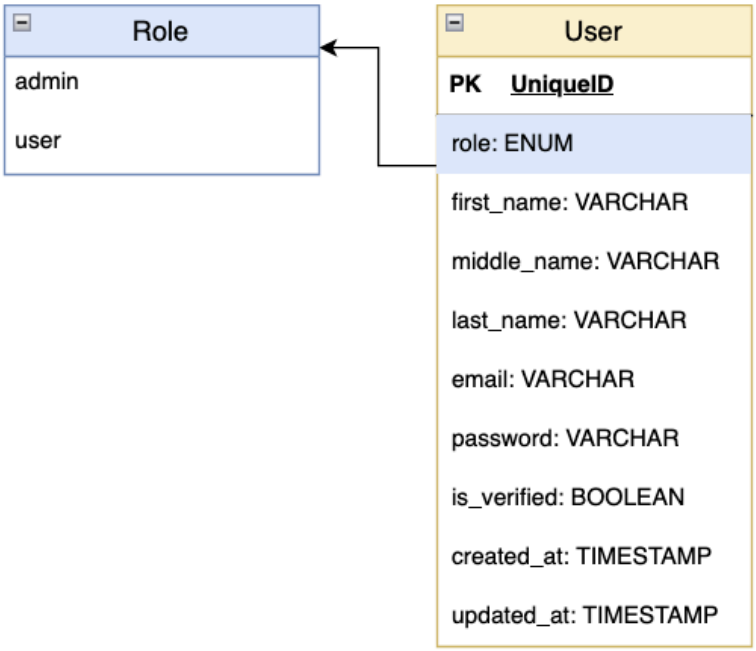


Разделение на микросервисы:

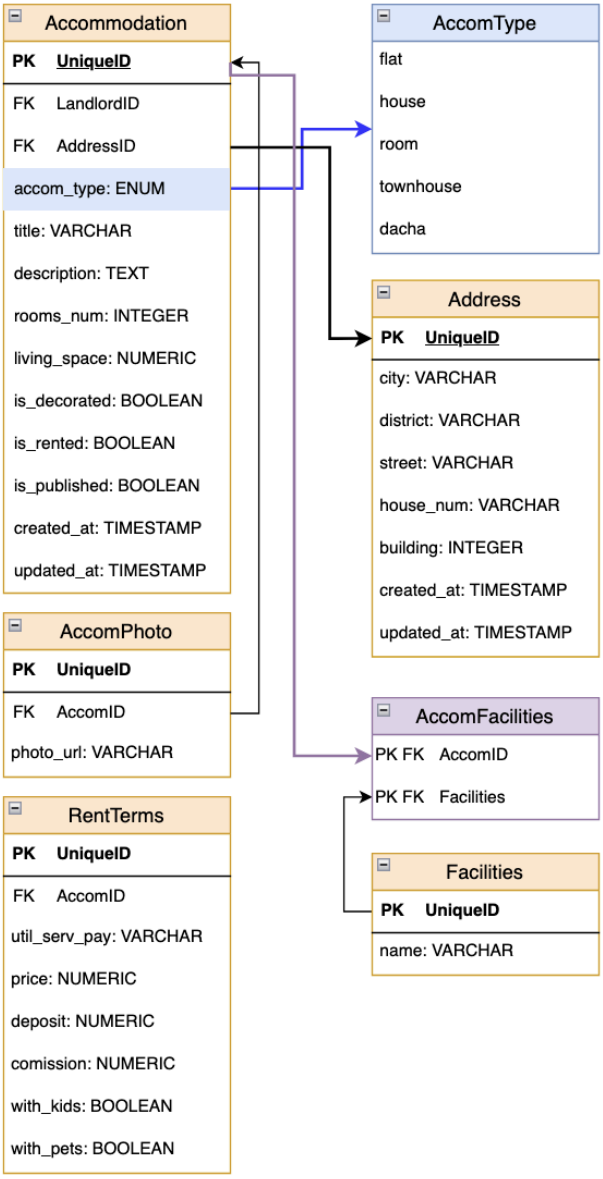


Выделенные сервисы:

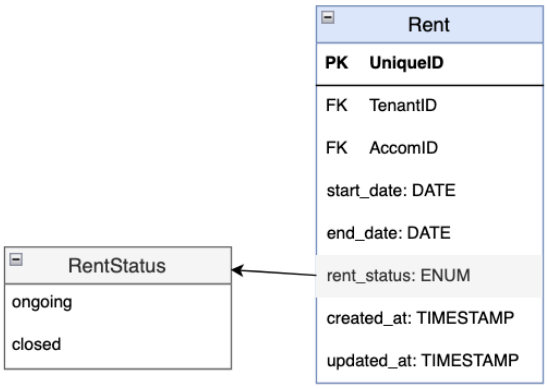
1. **User Service** (Identity & Access Management)
   a. Сущности: User, Role
   b. Ответственность: аутентификация, регистрация, управление профилями



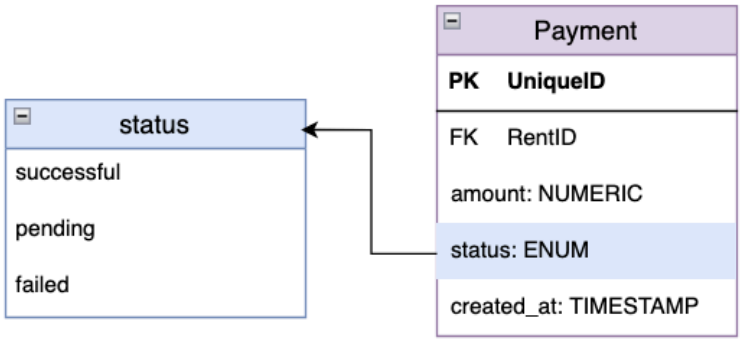
2. **Accommodation Service** (Property Management)
   a. Сущности: Accommodation, AccomType, Address, AccomPhoto, AccomFacilities, Facilities, RentTerms
   b. Ответственность: управление объектами недвижимости



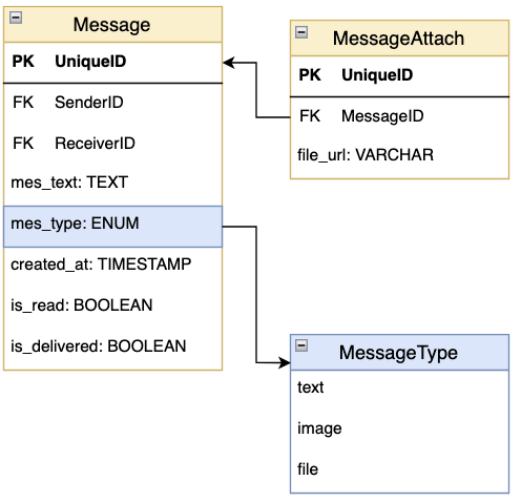
3. **Rent Service** (Rental Management)
   a. Сущности: Rent, RentStatus
   b. Ответственность: управление арендными соглашениями



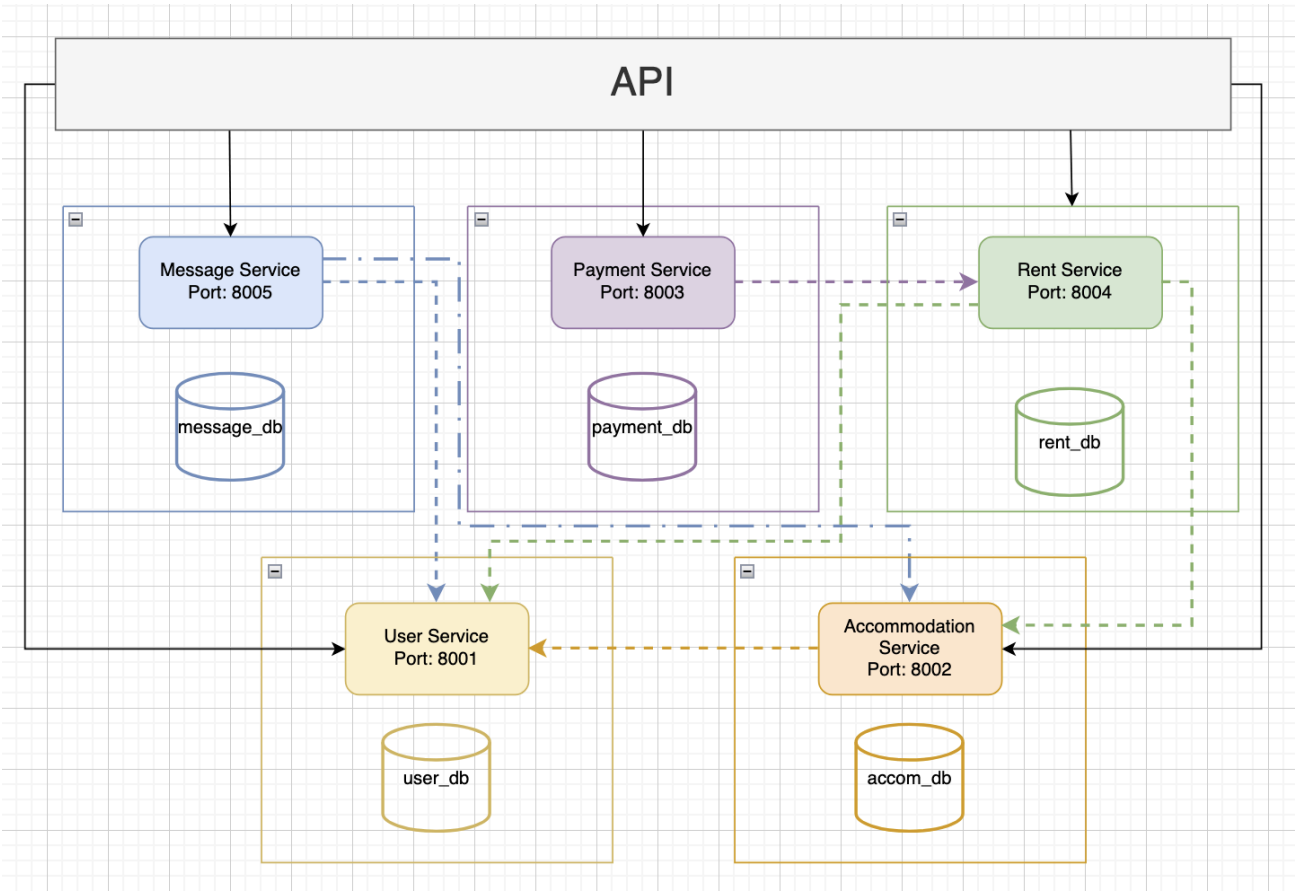
4. **Payment Service** (Payment Processing)
   a. Сущности: Payment, status
   b. Ответственность: обработка платежей



5. **Message Service** (Communication)
   a. Сущности: Message, MessageType, MessageAttach
   b. Ответственность: обмен сообщениями между пользователями



Взаимосвязь между микросервисами:



Users

GET /users/batch

GET /users/landlord/{id}

GET /users/tenant/{id}

GET /users/{id}/validate

GET /users/batch: Получить нескольких пользователей по списку ID

401 Unauthorized – отсутствует или невалиден JWT-токен сервиса
500 Internal Server Error – внутренняя ошибка при обработке запроса

GET /users/{id}/validate: Проверить, существует ли пользователь и активен ли он для транзакций

404 Not Found – пользователь не найден или неактивен
401 Unauthorized – ошибка аутентификации

GET /users/landlord/{id}: Получить данные пользователя в роли арендодателя

404 Not Found – пользователь не найден или не имеет роли арендодателя
401 Unauthorized – ошибка аутентификации

GET /users/tenant/{id}: Получить данные пользователя в роли арендатора

404 Not Found – пользователь не найден или не имеет роли арендатора
401 Unauthorized – ошибка аутентификации

Accommodations

GET	/accommodations/batch
GET	/accommodations/landlord/{landlordId}
GET	/accommodations/{id}
GET	/accommodations/{id}/availability
GET	/accommodations/{id}/rent-terms
PATCH	/accommodations/{id}/update-status

GET /accommodations/batch: Получить несколько объектов недвижимости по списку ID

401 Unauthorized – невалидный или отсутствующий токен
500 Internal Server Error – внутренняя ошибка при обработке запроса

GET /accommodations/{id}: Получить полную информацию об объекте недвижимости

404 Not Found – объект не найден

GET /accommodations/{id}/availability: Проверить доступность объекта на указанные даты

404 Not Found – объект не найден
400 Bad Request – некорректный формат дат или логическая ошибка (end_date < start_date)

GET /accommodations/{id}/rent-terms: Получить условия аренды

404 Not Found – объект не найден

GET /accommodations/landlord/{landlordId}: Получить все объекты конкретного арендодателя

401 Unauthorized – ошибка аутентификации

PATCH /accommodations/{id}/update-status: Обновить статус аренды объекта (is_rented)

404 Not Found – объект не найден
403 Forbidden – у сервиса нет прав на изменение статуса
400 Bad Request – некорректное значение в теле запроса

Rents

POST	/rents
GET	/rents/accommodation/{accomId}/active
GET	/rents/batch
GET	/rents/user/{userId}
GET	/rents/validate
GET	/rents/{id}
PATCH	/rents/{id}/status

GET /rents/batch: Получить несколько договоров аренды по списку ID

401 Unauthorized – ошибка аутентификации

GET /rents/{id}: Получить детали конкретного договора аренды

404 Not Found – договор не найден

POST /rents: Создать новый договор аренды (валидация дат и участников)

400 Bad Request – невалидные данные: отсутствуют обязательные поля, некорректные даты
409 Conflict – даты пересекаются с существующим договором на этот же объект

404 Not Found – не найден пользователь (tenant) или объект (accommodation)

GET /rents/validate: Проверить, можно ли арендовать объект на указанные даты (поиск конфликтов)

400 Bad Request – некорректный формат запроса

/rents/accommodation/{accomId}/active: Получить все активные договоры для объекта недвижимости

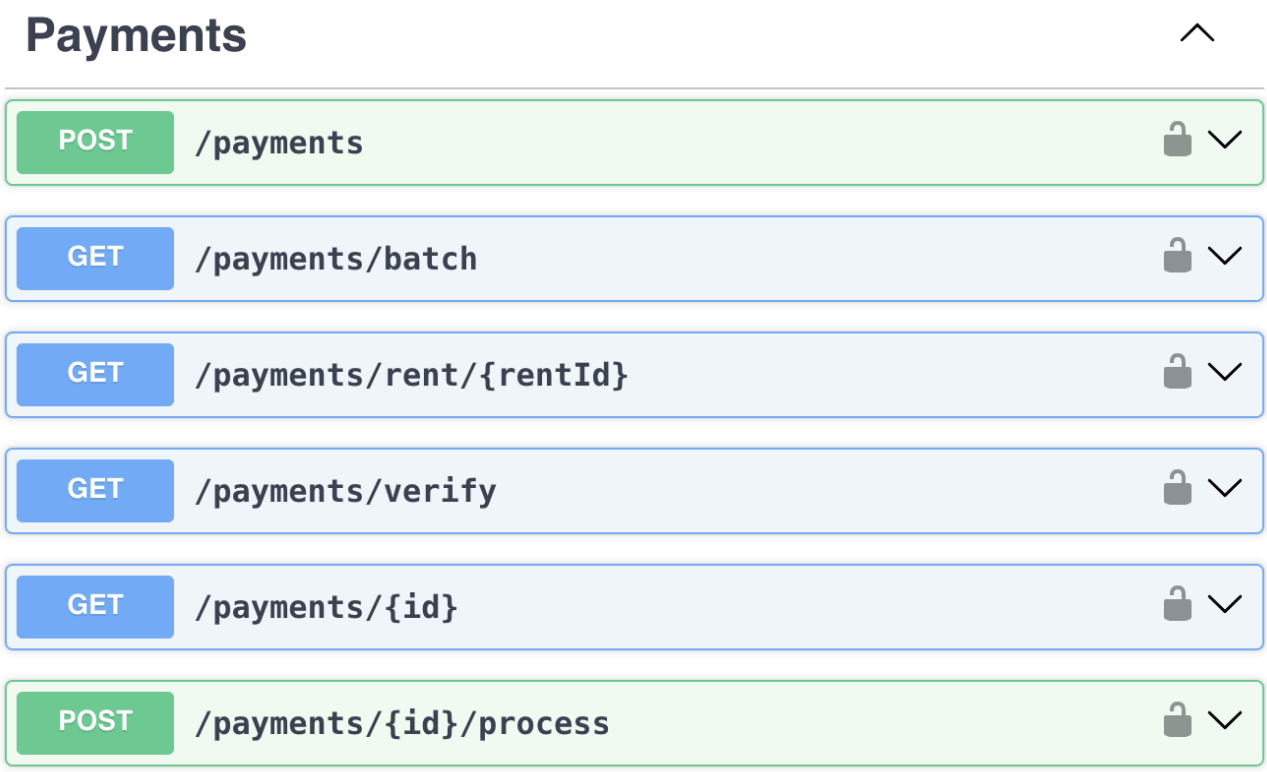
404 Not Found – объект не найден

/rents/user/{userId}: Получить все договоры, где пользователь – арендатор или арендодатель

400 Bad Request – невалидное значение параметра role

/rents/{id}/status: Обновить статус договора (ongoing / closed)

404 Not Found – договор не найден
400 Bad Request – невалидное значение статуса



GET /payments/batch: Получить несколько платежей по списку ID

404 Not Found – платеж не найден
401 Unauthorized – ошибка аутентификации

GET /payments/{id}: Получить детали конкретного платежа

404 Not Found – платеж не найден
401 Unauthorized – ошибка аутентификации

POST /payments: Создать новый платеж для договора аренды

400 Bad Request – некорректные данные: отрицательная сумма, отсутствует rent_id
404 Not Found – договор аренды не найден

GET /payments/rent/{rentId}: Получить все платежи по конкретному договору аренды

404 Not Found – договор не найден (опционально)

GET /payments/verify: Верифицировать оплату: сравнить ожидаемую и фактически выплаченную сумму

400 Bad Request – некорректные входные данные

POST /payments/{id}/process: Инициировать обработку платежа

400 Bad Request – платёж не может быть обработан
404 Not Found – платеж не найден

Messages

POST	/messages
GET	/messages/accommodation/{accomId}
GET	/messages/batch
GET	/messages/conversation
GET	/messages/{id}

GET /messages/batch: Получить несколько сообщений по списку ID

401 Unauthorized – ошибка аутентификации

GET /messages/{id}: Получить детали конкретного сообщения

404 Not Found – сообщение не найдено

POST /messages: Отправить новое сообщение между пользователями

400 Bad Request – пустой текст, отправитель = получателю, невалидный тип сообщения

GET /messages/conversation: Получить историю переписки между двумя пользователями

400 Bad Request – некорректные параметры пагинации (отрицательный limit/offset)

GET /messages/accommodation/{accomId}: Получить все сообщения, связанные с объектом недвижимости

400 Bad Request – некорректное значение limit

Пример с Rent Service:

Клиент → POST-запрос на создание аренды (/rents), при этом передает идентификатор арендатора, идентификатор жилья и даты.

API Gateway → перенаправляет запрос в Rent Service

Rent Service:
1. Проверяет арендодателя: отправляет запрос в User Service (/internal/users/validate с user_id=N). Проверяет, существует ли пользователь и подтвержден ли он, и возвращает успешный ответ с данными пользователя.

2. Проверяет доступность жилья: запрашивает в Accommodation Service (/internal/accommodations/K/availability) наличие жилья на указанные даты. Accommodation Service проверяет статус жилья, делает запрос к базе данных на предмет активных аренд и возвращает положительный ответ.

3. Получает данных арендодателя: запрашивает в User Service информацию о владельце жилья (арендодателе) через GET /internal/users/landlord/N. User Service возвращает эти данные.

4. Проводит транзакцию: Rent Service открывает транзакцию, делает вставку записи об аренде в таблицу rents, обновляет статус жилья (is_rented = true) и фиксирует транзакцию.

5. Отправка события: Публикует событие rent.created в Message Queue.

6. Формирование ответа: Возвращает API Gateway ответ с кодом 201 Created

API Gateway передает этот ответ клиенту.

Примеры событий:

User Service:

user.registered – новый пользователь зарегистрировался
user.verified – email/телефон подтверждён
user.deleted – пользователь удалил аккаунт

user.role.upgraded – добавлена новая роль (например, стал арендодателем)
user.profile.updated – изменены персональные данные
user.password.changed – смена пароля
user.logged.in – успешный вход
user.logged.out – выход из системы

Accommodation Service:

accommodation.created – добавлен новый объект недвижимости
accommodation.updated – изменены характеристики или условия
accommodation.deleted – объект удален
accommodation.published – объект опубликован и доступен для аренды
accommodation.unpublished – снят с публикации
accommodation.available – снова доступен
accommodation.photo.added – добавлена новая фотография
accommodation.photo.removed – фотография удалена
accommodation.price.changed – изменена стоимость аренды

Rent Service:

rent.created – создан новый договор аренды
rent.updated – изменены даты или условия
rent.cancelled – аренда отменена
rent.started – аренда началась
rent.completed – аренда успешно завершена

Payment Service:

payment.created – платеж создан (ожидает обработки)
payment.processed – платеж успешно проведен
payment.failed – платеж не удался (недостаточно средств, ошибка шлюза)

Message Service:

message.sent – отправлено новое сообщение
conversation.started – начат новый диалог между пользователями
message.attachment.added – добавлено вложение

Вывод

Монолитное приложение было разделено на микросервисы.